

Value-at-Risk, Three Ways

Three standard recipes for the same number — and a backtest that rejects all of them.

How much could we lose tomorrow? Every trading desk has to answer that question before the market opens, and Value-at-Risk is the industry's standard answer: one number that says “on 99 days out of 100, the loss stays below this line.” The catch is that there are three standard recipes for computing that line — and they disagree exactly when it matters, on the bad days. We compute all three on fifteen years of real DAX data, put every one of them on trial out of sample, and then recompute the historical quantile with Polars and DuckDB to show what the modern data stack changes (answer: the scaling, never the number).

PROJECT CARD

CATEGORY

Risk Management

LIBRARIES

NumPy · SciPy · Pandas · Polars · DuckDB

ASSETS

DAX (^GDAXI)

TIMEFRAME

Jan 2010 – Dec 2024

USE

Setting and defending a daily loss limit — and proving it holds up

- 01 Download DAX (^GDAXI) closes, 2010–2024 (yfinance)
- 02 Historical simulation: empirical 1% and 5% quantiles
- 03 Parametric: normal, then Student-t with fitted df
- 04 Monte Carlo: 200,000 seeded draws from the fitted t
- 05 Backtest all four with a rolling 250-day window + Kupiec POF
- 06 Re-do the quantile in Polars expressions and DuckDB SQL

1. The number, and the data

Line up fifteen years of daily returns from worst to best. **VaR** at 99% is a marker planted one percent of the way in from the bad end: the loss you do not expect to exceed on 99 days out of 100. Formally it is the $(1-\alpha)$ -quantile of returns, negated — but “the flood line” is the right mental picture.

CVaR (expected shortfall) answers the question a good risk manager asks next: **and when the water does cross the line, how deep does it get?** It is the average loss given a

breach, and we report it alongside every method because it is the number you actually want for limits. Our laboratory is the DAX from 2010-01-04 to 2024-12-30: 3,805 daily returns spanning the euro crisis, COVID and the 2022 energy shock. The worst day in the sample is 2020-03-12 at -12.24% — keep that number in mind while the normal distribution tells you it is impossible.

DEFINITIONS

$VaR_\alpha = -q_{\alpha}(\text{returns})$ · $CVaR_\alpha = -E[r \mid r \leq q_{\alpha}]$ — both quoted as positive percentages of portfolio value, one-day horizon throughout.

2. Historical simulation

The first recipe refuses to assume anything. Sort the returns, walk one percent of the way in from the worst end, read off the number — that is the entire method. On the full sample the 99% VaR is 3.42% and the 99% CVaR — the average of the worst 38 days — is 4.60%. At 95% the pair is 1.94% / 2.93%.

```
import numpy as np

def hist_var_cvar(x, alpha=0.99):
    q = np.quantile(x, 1 - alpha)      # 1% quantile of returns
    return -q, -x[x <= q].mean()      # VaR, CVaR

hist_var_cvar(ret.values)              # (0.034169, 0.045981)
```

That honesty cuts both ways. The method treats a sleepy 2017 Tuesday exactly like March 2020, and it has no imagination: it cannot warn you about any loss larger than one it has already lived through. History is its only teacher — a problem, because the worst day of the next fifteen years is under no obligation to have a precedent in the last fifteen.

3. Parametric: normal, then Student-t

The second recipe trades honesty for a formula: assume returns follow a known distribution, and VaR drops straight out of its quantile function. Assume a normal ($\mu = 0.039\%$, $\sigma = 1.228\%$ daily) and the 99% VaR comes out at 2.82% — roughly 60bp **below** the empirical quantile. That shortfall is the price of the assumption: the bell curve simply does not believe in days as bad as the ones the DAX has actually had. So keep the formula but change the bell. Refit with a Student-t — the normal's heavy-tailed cousin — and maximum likelihood hands you $df = 3.22$: violently non-Gaussian tails. The t's 99% VaR of 3.40% lands almost exactly on the historical number, and its CVaR (5.13%) is fatter still. (One desk convention worth knowing: at the one-day horizon many shops zero out μ entirely — at 0.039% a day it is noise against σ , and estimating it adds error without information. We keep it for completeness.)

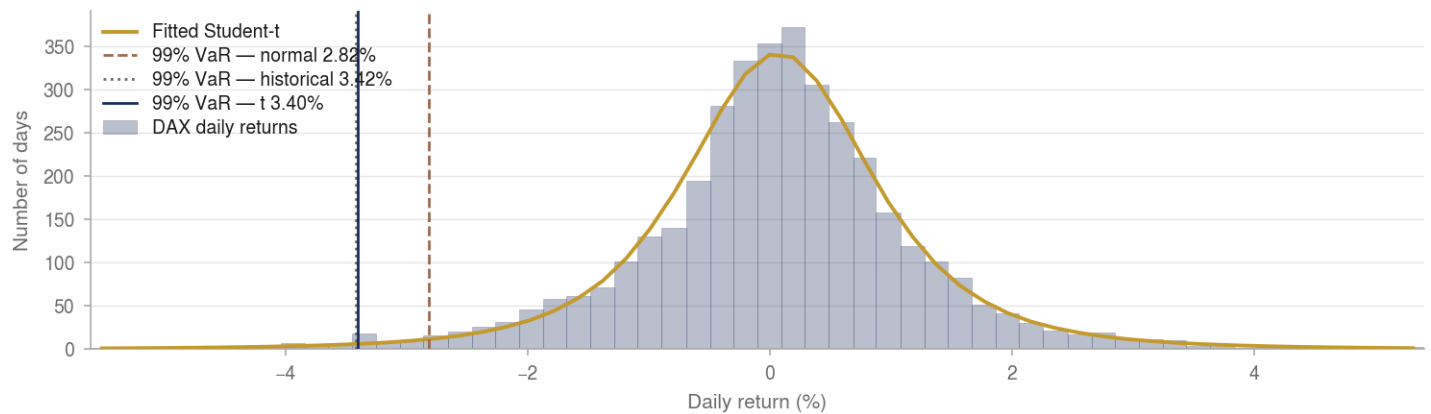
```
from scipy import stats

mu, sd = ret.mean(), ret.std(ddof=1)
z = stats.norm.ppf(0.01)
var_normal = -(mu + sd * z)          # 2.82%
```

```
df, loc, scale = stats.t.fit(ret.values)      # df = 3.22
var_t = -(loc + scale * stats.t.ppf(0.01, df)) # 3.40%
```

3,805 DAX daily returns — fitted Student-t overlay, 99% VaR markers

The t and historical markers nearly coincide; the normal sits well inside them

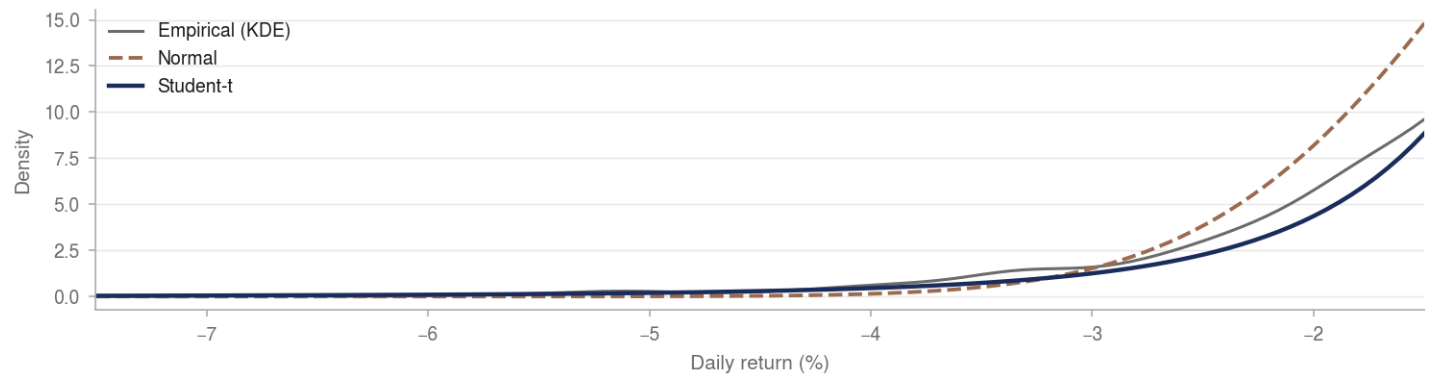


Source: Author calculations, from the article's computed results. DAX daily returns, January 2010 – December 2024.

Where did the normal lose those 60 basis points? Watch it happen. Below are the three densities over the loss region from -7.5% to -1.5%: the normal (amber) runs out of probability almost immediately — by -3% it has essentially declared such days impossible — while the fitted t tracks the kernel estimate of the real data the whole way down. The 60bp gap in VaR is this picture, integrated.

Left-tail densities, -7.5% to -1.5% — normal vs Student-t vs empirical (KDE)

The normal density collapses toward zero beyond -3% while the t stays close to the empirical estimate



Source: Author calculations, from the article's computed results.

4. Monte Carlo

The third recipe replaces formulas with brute force: simulate 200,000 one-day scenarios from the fitted t (seed 42) and read the tail of the simulation as if it were history you simply have not lived yet. On a single linear asset this is a correctness check more than a method — it must reproduce the analytic t to Monte-Carlo error, and it does: 3.39% vs the analytic 3.40%. So why keep it? Because simulation does not care whether a closed form exists. The moment the book contains options, path dependence, or anything else with no analytic quantile, it is the only recipe still standing.

```
rng = np.random.default_rng(42)
draws = loc + scale * rng.standard_t(df, 200_000)

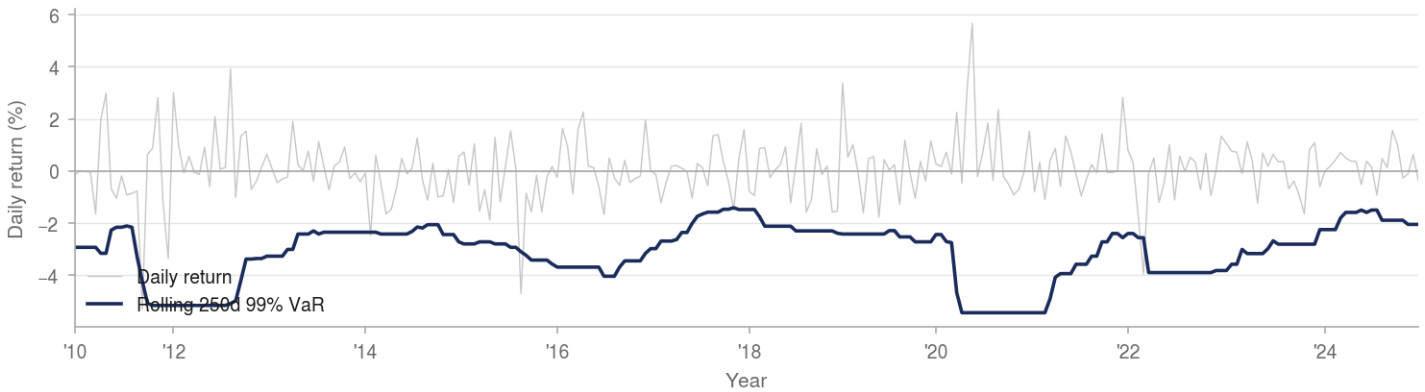
var_mc = -np.quantile(draws, 0.01)      # 3.39%
cvar_mc = -draws[draws <= np.quantile(draws, 0.01)].mean() # 5.08%
```

5. The backtest decides

Three recipes, three different answers — so who is right? Here is the beautiful thing about VaR: it is a **falsifiable forecast**. If a 99% VaR is honest, tomorrow's loss should exceed it on about 1% of days — no more, no fewer — and we can simply count. We re-estimate each method on a rolling 250-day window (t and Monte Carlo refit every 20 days, desk-style), forecast one day ahead — 3,555 out-of-sample forecasts per method — and test the breach count with Kupiec's proportion-of-failures likelihood ratio.

DAX daily returns vs the rolling 250d historical 99% VaR line, 2010-12-23 → 2024-12-30

The threshold plunges after 2011, 2020 and 2022 as crash days enter the estimation window



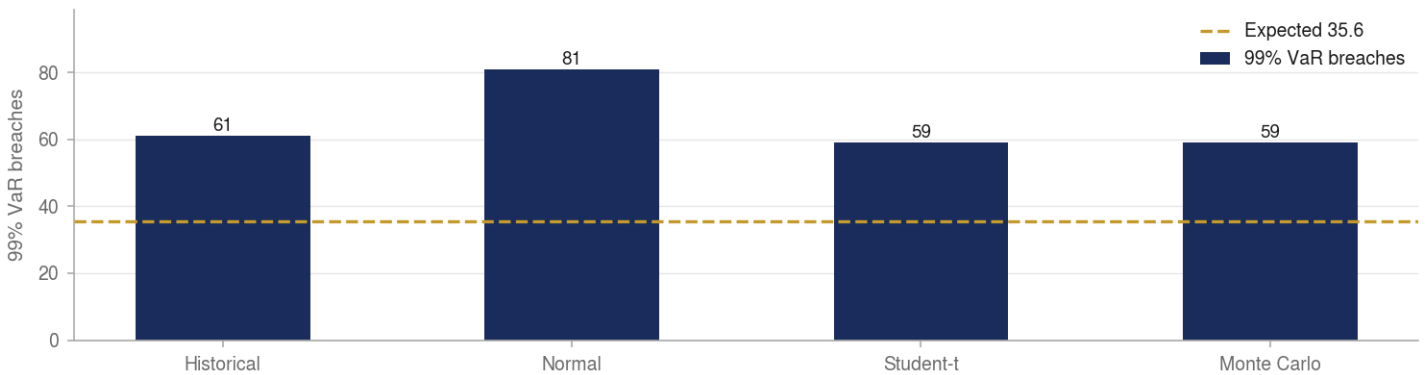
Source: Author calculations, from the article's computed results.

The tell-tale shape: the VaR line jumps **after** each crisis enters the window and relaxes as it leaves. That lag is why breaches cluster — the first breach of the backtest lands on 2011-03-15, deep in the euro crisis, and the worst runs come in March 2020 before the window has learned what COVID is.

Method	99% VaR	99% CVaR	Breaches (exp. 35.6)	Kupiec LR	p-value
Historical	3.42%	4.60%	61	15.16	9.9e-5
Parametric normal	2.82%	3.24%	81	43.10	5.2e-11
Parametric t (df 3.22)	3.40%	5.13%	59	13.04	0.0003
Monte Carlo (t)	3.39%	5.08%	59	13.04	0.0003

Out-of-sample 99% VaR breaches per method, 3,555 forecasts — dashed line = expected count

Every method breaches more often than its own 1% promise; the normal by the widest margin



Source: Author calculations, from the article's computed results.

Read it honestly: **every** bar clears the expected line — even the best method understated its own failure rate. The t roughly halves the normal's excess, but Kupiec rejects all four at the 1% level. And Kupiec is the lenient examiner: it only counts breaches, never asking **when** they arrived. One look at the chart shows them arriving in volatility clusters, which is exactly the

pattern Christoffersen’s (1998) independence test is built to punish and the pattern behind Basel’s traffic-light backtest zones. The diagnosis is structural, not a fixable bug: a rolling window only ever looks backwards, so it is late to every regime change by construction. That is not a reason to despair; it is the empirical case for conditional risk models — a GARCH filter rescales the tail to **today’s** volatility, and filtered historical simulation is the desk standard for precisely this failure mode.

6. The same quantile in Polars and DuckDB

Strip the finance away and historical VaR is one line of data engineering: a quantile over a column. That makes it a perfect specimen for a question every quant team eventually asks — does the modern data stack change the answer? **Polars** evaluates a lazy expression pipeline over the CSV; **DuckDB** runs SQL straight against the file. Neither needs the data in pandas, and both stream — the identical two lines still work when “one index, fifteen years” becomes “every book in the firm, tick by tick”.

```
import polars as pl
import duckdb

q_pl = (pl.scan_csv("dax_returns.csv")
        .select(pl.col("ret").quantile(0.01, interpolation="linear"))
        .collect()
        .item())                                # -0.034169015644486

q_db = duckdb.sql(
    "SELECT quantile_cont(ret, 0.01) FROM read_csv('dax_returns.csv')"
).fetchone()[0]                                # -0.034169015644486
```

Engine	1% quantile of returns	diff vs pandas
pandas / NumPy	-0.034169015644486	—
Polars 1.43 (lazy)	-0.034169015644486	0.0e+0
DuckDB 1.5	-0.034169015644486	0.0e+0

All three agree to the last printed digit (max absolute difference 0.0e+0, well inside the 1e-12 tolerance the pipeline asserts). On 3,805 rows the timings are dominated by engine start-up — NumPy answers in a fraction of a millisecond, Polars in a few, DuckDB in tens — so at this size the engines buy you nothing but ergonomics. The crossover comes when the file stops fitting in memory: the pandas version dies, the Polars and DuckDB versions do not change by a character.

PRACTITIONER TAKE

A Student-t beats a normal for VaR almost always — here it is the difference between 81 and 59 breaches on the same data. Prefer CVaR over VaR for limits: it sees how bad the breach is, and at 99% it runs more than a full point above VaR on the DAX (3.42% → 4.60% historical, 3.40% → 5.13% under the t). And never ship a VaR you have not backtested — every method in this article looked respectable until Kupiec counted its breaches.

REFERENCES

1. Jorion, P. *Value at Risk: The New Benchmark for Managing Financial Risk*. McGraw-Hill.
2. Kupiec, P. (1995). Techniques for Verifying the Accuracy of Risk Measurement Models. *Journal of Derivatives*, 3(2).
3. Christoffersen, P. (1998). Evaluating Interval Forecasts. *International Economic Review*, 39(4), 841–862.
4. McNeil, A., Frey, R. & Embrechts, P. *Quantitative Risk Management: Concepts, Techniques and Tools*. Princeton University Press.
5. Companion notebook: `var-three-ways.ipynb` — reproduces every number from raw data (seed 42), including the Polars and DuckDB cells.